



THE CLOUD & AI SUMMIT • 2026

Sept 30 - Oct 2, 2026

Ameristar • St. Charles, MO

Thank you to our **sponsors.**

PREMIUM EXHIBITORS



EXHIBITORS



EXPERIENCE SPONSORS





THE CLOUD & AI SUMMIT 2026 · ST. CHARLES, MO · SEPT 30 – OCT 2

When Logs Tell the Whole Story

OpenTelemetry for Beginners — including your AI calls

Barret Blake · Architect, NimblePros

#CloudAISummit | #OTel | #Observability



WHO AM I

Barret Blake



Barret Blake

Architect · NimblePros



A few things about me

-  Microsoft MVP — .NET
-  Husband & Father
-  Coder, Blogger & Speaker
-  Mentor
-  Gamer
-  Model Railroader & LEGO Enthusiast
-  Buckeyes & Browns Fan

THE PROBLEM

Production failures are inevitable

What happens next is up to you.



No observability

Guessing in the dark. Each incident is a mystery. MTTR spirals.



Scattered logs

Logs exist but tell disconnected stories. Hard to correlate across services.



Structured telemetry

A clear, connected story. Logs trace the path from failure to root cause.

The difference between a quick fix and a prolonged outage **often comes down to whether your logs tell a clear, connected story.**

What is OpenTelemetry?

An open-source, language-agnostic, vendor-neutral framework for application observability — providing APIs, SDKs, and tools to capture traces, metrics, and logs.



Open Source

Backed by the CNCF. Community driven and widely adopted.



Language Agnostic

.NET, Java, Go, JS, Python, Rust, Ruby & more.



Vendor Neutral

Send your data to ANY compatible observability platform.

The industry standard for cloud-native observability.

The Three Signals of OTel



Traces

The full path of a request through your system — entry point to database and back.



Metrics

Numeric measurements over time — request counts, error rates, latency percentiles.



Logs

Timestamped records of what happened and why. The base unit of all telemetry.



Baggage — not a signal, but cross-cutting context: key-value data propagated across all spans and service boundaries within a trace.



Coming soon: Profiles — an emerging fourth signal (public alpha, 2026) answering “which code is burning CPU / memory?”

Traces: Following the Request

A trace records the complete path of a request through your application.

- ✓ Each unit of work = a Span
- ✓ Spans nest into parent / child trees
- ✓ Captures timing for every step
- ✓ Spans across services share a TraceId
- ✓ Critical for cross-service debugging

TRACE WATERFALL VIEW

LaunchMission



POST /api/missions/{id:int}/launch



DATA sqlite main



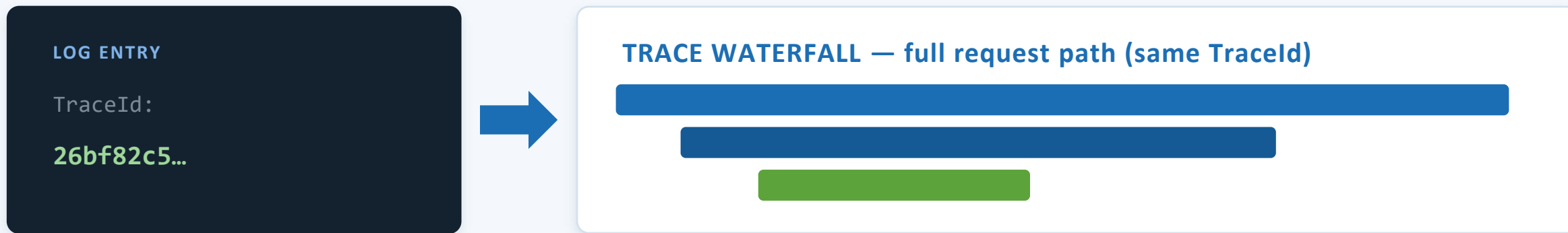
← Time →



Real-world impact: Analyzing trace waterfalls on a client project revealed several performance bottlenecks — a **10–40% runtime reduction** across key workflows.

Trace Context: Connecting the Dots

The same TraceId and SpanId appear in both your logs AND your trace spans.



One click to context

Jump from a log line directly to the full trace waterfall in your platform.



Cross-service correlation

The same TraceId flows through all services — find a user's request across microservices instantly.



No manual correlation

OTel propagates trace context automatically via HTTP headers and messaging metadata.

Writing Your Own Span

```
// 1 – one named source per service, created once
static readonly ActivitySource Source =
    new("MissionControl.Telemetry");

// 2 – the span lives as long as the using block
using var activity =
    Source.StartActivity("LaunchMission");

// 3 – tag what you'll filter on at 2 a.m.
activity?.SetTag("mission.name", mission.Name);
activity?.SetTag("mission.commander", commander);

// 4 – register it, or nothing exports
.WithTracing(t => t
    .AddSource("MissionControl.Telemetry"))
```

1

ActivitySource

A named factory — like Meter, but for spans.

2

StartActivity

In .NET a span is called an Activity. The using block ends it and records the duration.

3

SetTag

Identifiers, not payloads. Keep the ?. — it's null when nobody is listening.

4

AddSource — or it's silent

An unregistered source compiles, runs, and exports nothing. No error, no warning. It is always this.

Metrics: Measuring What Matters



Error-Rate Alerting

Don't alert on every failure — you'll train your team to ignore alerts. Use a metric counter and fire only when a threshold is crossed (e.g. 20+ failures in 60 seconds).



Code Usage Analysis

Before investing in a refactor, use metrics to prove it's worth it. If a 1,000-line class runs hundreds of times per minute, that data is your business case.

```
// Custom counter setup
public MissionSimulator(IMeterFactory meterFactory)
{
    var meter =
meterFactory.Create(MissionTelemetry.MeterName);

    _warp = meter.CreateGauge<double>("mission.warp_speed",
unit: "warp", description: "Current warp factor of an in-
flight mission.");
    _warp.Record(run.WarpSpeed, shipTag);

    _events = meter.CreateCounter<long>("mission.events",
unit: "{event}", description: "In-flight mission events by
type.");
    _events.Add(1, shipTag, new KeyValuePair<string,
object?>("event", evt));
}
```



Metric output: 3 launches for Enterprise, 1 for Voyager.

Logs: The Story Behind the Numbers

“Not all logs are events, but all events are logs.” — OpenTelemetry docs



Automatically shipped

OTel exports the log records you already create — the exporter handles it, no special wiring.



Structured output

OTel turns any log format into a structured record: timestamp, severity, body, attributes.



Correlated with traces

Every log within a trace carries the TraceId and SpanId — connecting logs to the request.

When you need to understand what went wrong and why — logs are where you look.

Anatomy of an OTEL Log Record

```
LogRecord.Timestamp: 2026-09-10T09:14...
LogRecord.TraceId: 26bf82c5...
LogRecord.SpanId: c0da8620...
LogRecord.TraceFlags: Recorded
LogRecord.CategoryName: Program
LogRecord.Severity: Info
LogRecord.Body: Mission launch
    requested: {MissionName}...
LogRecord.Attributes:
    MissionName: Enterprise
    Status: In Flight
```

Field	Purpose
Timestamp	When the event occurred
TraceId	Links log to its trace
SpanId	Specific span within the trace
TraceFlags	W3C standard trace flags
CategoryName	.NET ILogger category
Severity	Log level (Info, Warn, Error...)
Body	The log message
Attributes	Structured key-value data

```
logger.LogInformation("Mission launch requested: {MissionName}, commander: {Commander}, priority: {Priority}, status: {Status}", mission.Name, commander, priority, mission.Status);
```

Choosing the Right Log Level

Level	Use When...	Recommended For
Trace	Highly detailed debugging. Very high volume. Dev only.	Never in production
Debug	Diagnostic details for development and debugging.	Dev / staging only
Information	Normal operations. Application milestones, key events.	Production — selectively
Warning	Unexpected but recoverable. Worth monitoring.	Always in production
Error	Operation failed. Needs attention; app still running.	Always in production
Critical	Catastrophic failure. Immediate action required.	Always + alerting

Common Logging Mistakes to Avoid



Over-Logging

Problem: Logging everything buries the signal in noise — and drives up cost on commercial platforms.

Fix: Log meaningful events at appropriate levels. Filter dev-only logs from production.



Unstructured Messages

Problem: String concatenation (“Error for user ” + id) can't be queried, filtered, or aggregated.

Fix: Use structured logging: `LogError(“User {UserId} failed checkout”, id)` — queryable attributes.



Missing Context

Problem: “Operation failed” is useless without which operation, which user, and what data.

Fix: Always include identifiers: user IDs, record IDs, operation names. Failures are data-specific.



Alert-Fatigue Loops

Problem: Alerting on every failure trains your team to build rules that ignore alerts entirely.

Fix: Use metric thresholds, not per-event alerts. Alert on patterns, not occurrences.

Baggage: Context That Travels

Key-value context that propagates automatically across all spans, services, and HTTP headers within a trace.

```
// Set baggage at request start
Baggage.SetBaggage(
    "mission.commander", "Picard");
Baggage.SetBaggage(
    "mission.priority", "High");

// Read anywhere in the trace
var commander = Baggage
    .GetBaggage("mission.commander");
```



Good uses for Baggage

- User ID for correlation
- Tenant ID in multi-tenant apps
- Source IP address
- Feature-flag or A/B testing group
- Request source / channel



Security warning: Baggage is added to HTTP headers and is readable by any receiving service. **Never put PII, passwords, or sensitive data in baggage.**

Setting Up OTel in .NET

Step 1 — Add NuGet Packages



OpenTelemetry.Extensions.Hosting



OpenTelemetry.Instrumentation.AspNetCore



OpenTelemetry.Exporter.OpenTelemetryProtocol



That's it for basic setup!

OTel auto-instruments your ASP.NET Core routes and starts collecting logs, traces, and metrics immediately. For Aspire projects it's included in the ServiceDefaults template — no manual setup.

```
// Step 2 – configure in Program.cs
builder.Logging.AddOpenTelemetry(o => o
    .SetResourceBuilder(ResourceBuilder
        .CreateDefault()
        .AddService("app-logger"))
    .AddOtlpExporter());

builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r
        .AddService("app-logger"))
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(m => m.AddOtlpExporter());
```

Exporting to Production Platforms



Self-Hosted

Jaeger — Distributed tracing

Grafana + Loki + Prometheus — Metrics, logs & dashboards

Elastic Stack — Search-first observability

Pros: control costs, data ownership. **Cons:** you own maintenance.



Commercial SaaS

Datadog — Full-stack observability

Grafana Cloud — Hosted Grafana + alerting

Azure App Insights — Native .NET + Azure

New Relic — AI-powered incident response

Pros: managed, enterprise features. **Cons:** cost scales with data.

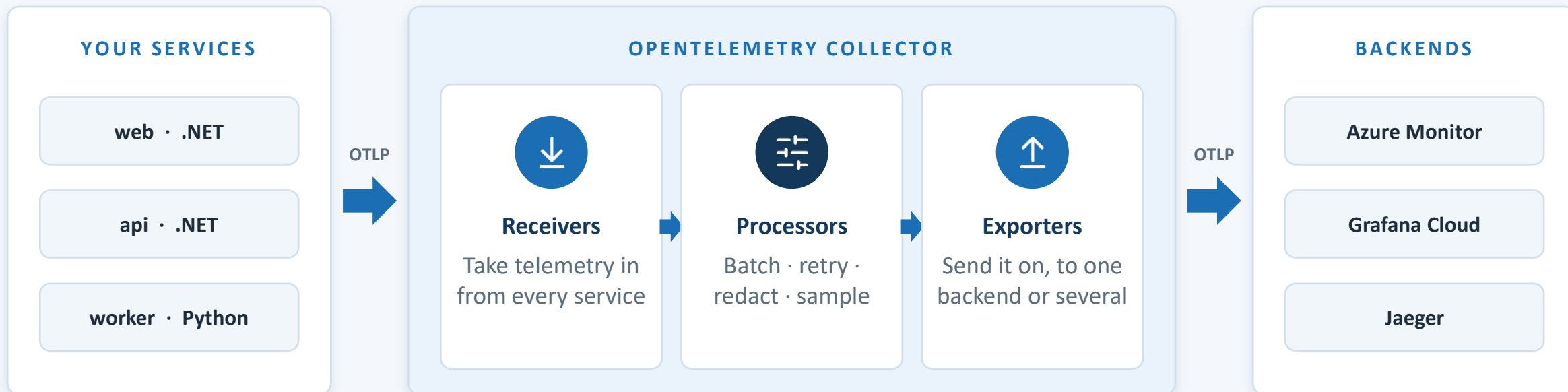


The OTEL advantage: portability

Everything speaks **OTLP**, the OpenTelemetry wire protocol. Route through the **OpenTelemetry Collector** and switching platforms is a config change — not a rewrite. Your data model stays the same.

The OpenTelemetry Collector

A vendor-neutral proxy between your apps and your backend — configured in one YAML file.



Batch & retry

Backend down? The Collector buffers. Your app doesn't.



Fan-out

Two backends at once. Migrate vendors without a deploy.



Scrub & redact

Mask or drop attributes before they leave your network.



Sample

Keep the interesting traces, drop the rest. Next slide.

Sampling: Keep the Interesting, Drop the Boring



Head sampling

AT THE START · IN THE SDK

Roll the dice on the TraceId: keep 10%

```
new TraceIdRatioBasedSampler(0.10)
```

One line, nearly free, every service agrees

Catch: it decides before anything has happened — it drops 90% of your errors too.



Tail sampling

AT THE END · IN THE COLLECTOR

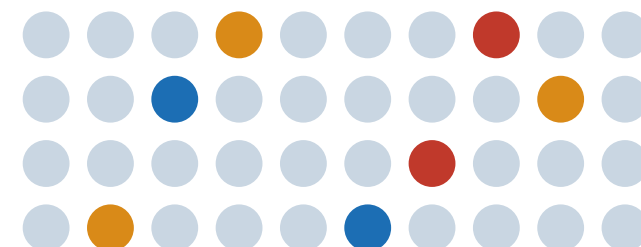
Keep **100%** of traces with an error

Keep **100%** of traces slower than 2 s

Keep **5%** of everything else

Catch: the Collector holds whole traces in memory while it waits. Real infrastructure.

40 REQUESTS IN → 7 TRACES KEPT



● error — 2 of 2 kept ● slow — 3 of 3 kept
● baseline — 2 kept ● boring — 33 dropped



Traces are sampled. Metrics are not.

Metrics are aggregated in-process before anything is dropped, so request counts and error rates stay exact and your threshold alerts still fire. **Counts come from metrics. Examples come from traces.**

Aspire: OTel Built In for Local Dev



Console View

Raw log output across all services in one pane.



Structured Logs

Filter and search by severity, category, or attribute.



Trace List

Browse all traces, filter by service or duration.



Waterfall View

Visual span timing — pinpoint slow code instantly.



Metrics

Runtime metrics and custom counters in one dashboard.



Multi-Language

Aspire 13 is now a multi-language platform — .NET, Python & JS/Node.



Don't deploy the Aspire dashboard to production. It exposes full debug logs with no auth. Use a dedicated observability platform in deployed environments.

Reusable Telemetry with ServiceDefaults

The ServiceDefaults shared project is the perfect home for reusable telemetry across your whole solution.



Custom Metric Counters

Define counters in ServiceDefaults via an AddMeter extension, then call counter.Add(1) in any endpoint. Metrics appear instantly in the dashboard.



Reusable Span Factory

A generic StartDatabaseSpan<T>() auto-tags span attributes from your entity's properties. One method, consistent tracing everywhere.

```
// ServiceDefaults: reusable span factory
public static Activity? StartDatabaseSpan<T>(
    this ActivitySource src, string name, T entity)
{
    var activity = src.StartActivity(
        name, ActivityKind.Internal);
    if (activity is null) return activity;
    // auto-tag entity properties...
    foreach (var p in typeof(T).GetProperties())
        activity.SetTag($"db.entity.{p.Name}",
                        p.GetValue(entity));

    return activity;
}
```



DEMO TIME

Mission Control

Launch a mission → watch traces, metrics, logs & baggage light up the Aspire dashboard in real time.

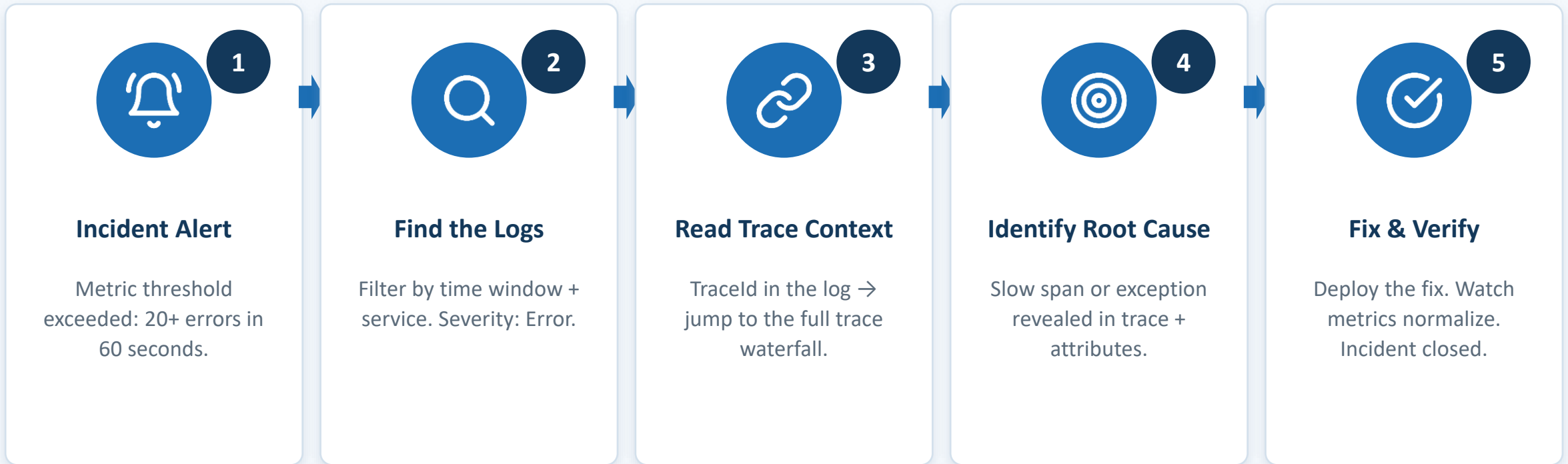
Traces

Metrics

Logs

Baggage

The Observability Stack in Action



Your model call is a black box

It breaks three things you were relying on.



Non-deterministic

Same input, different output. You cannot reproduce a bad answer by running it again — the trace is the only record it ever happened.



Slow, and not your fault

Milliseconds become seconds. It is now the slowest thing in your app, and the part you control least.



Priced per call

Every invocation has a variable cost. If you cannot see tokens, you find out the price when the invoice arrives.

You already know how to ask where the time went. **Now ask what the model did, how long it took, and what it cost — with the same three signals.**

The gen_ai Semantic Conventions

Attribute	What it records	Why you want it
gen_ai.operation.name	chat, embeddings, execute_tool	Which kind of call this was
gen_ai.provider.name	openai, azure.ai.openai, ollama	Compare vendors, keep one query
gen_ai.request.model	The model you asked for	Cost and latency, sliced by model
gen_ai.response.model	The dated build that answered	Evidence when behaviour shifts
gen_ai.usage.input_tokens	Tokens you sent	Half of the bill
gen_ai.usage.output_tokens	Tokens generated	The other half — priced higher

THE KEY IDEA

Instrument the Interface, Not the Vendor

Microsoft.Extensions.AI puts every provider behind one `IChatClient` — and `OpenTelemetry` attaches to the interface, so your telemetry never learns which vendor you picked.



One wrapper, any provider

OpenAI, Azure OpenAI, a local Ollama model, or a fake one for tests. The spans, attributes and metrics are identical. Switching models is a config change.



Registered like your own source

The chat source name goes into `AddSource` and `AddMeter` exactly like the `ActivitySource` you wrote earlier. Nothing new to learn.

```
// One wrapper. Any provider. Same telemetry.
IChatClient chat =
    new ChatClientBuilder(inner)
        .UseOpenTelemetry(
            sourceName: ChatSourceName,
            configure: o =>
                o.EnableSensitiveData = false)
        // content capture: off by default
        .Build();

// Register it like any other source:
.WithTracing(t => t.AddSource(ChatSourceName))
.WithMetrics(m => m.AddMeter(ChatSourceName));
```

Anatomy of an LLM Span

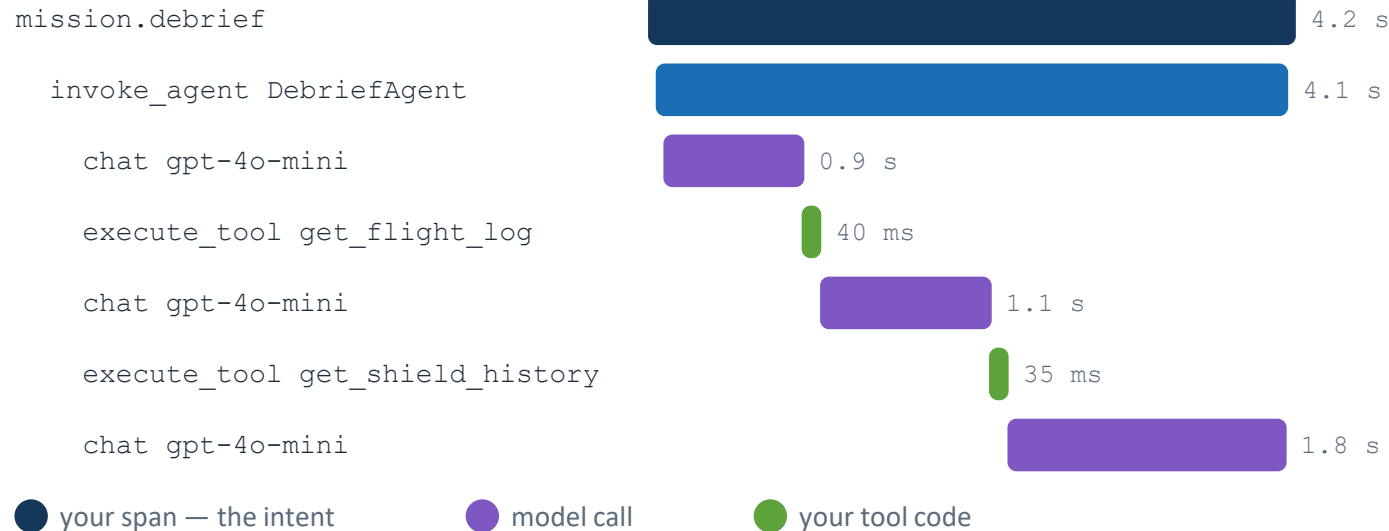
```
Span.Name:          chat gpt-4o-mini
Span.Duration:      836 ms
Span.Parent:        mission.debrief
gen_ai.operation.name: chat
gen_ai.provider.name: openai
gen_ai.request.model: gpt-4o-mini
gen_ai.response.model: gpt-4o-mini-2024-07-18
  finish_reasons: ["stop"]
gen_ai.usage:
  input_tokens: 1,284
  output_tokens: 312
```

Attribute	Why it matters
Span name	Operation + model, at a glance
Duration	Your slowest span, by far
operation.name	chat, embeddings, execute_tool
provider.name	Which vendor actually served it
request vs response	What you asked for vs what ran
finish_reasons	stop = done. length = truncated.
input_tokens	Tokens sent — half the bill
output_tokens	Generated — priced higher

// You wrote none of the gen_ai.* attributes above. The wrapper did.

When It's Not One Call: Agents & Tools

ONE CLICK · ONE TRACE



What the picture answers

- ✓ **How many model calls per click?**
Three — each with its own tokens.
- ✓ **What did one answer cost?**
Sum the tokens under the parent.
- ✓ **Model time or tool time?**
The model. Don't tune the tools.
- ✓ **Did it loop?**
Invisible to users. Obvious here.



Same conventions, two more operation names: `invoke_agent` and `execute_tool`. An agent is a distributed system where one service makes its own decisions — **the trace is the only record of what it decided.**

Tokens Are a Metric, and Metrics Are Money



Two histograms, free

`gen_ai.client.operation.duration` and `gen_ai.client.token.usage` come from the wrapper. Histograms, not averages — model latency has a long tail, and the tail is what your users feel.



One counter, yours

Add a token counter tagged by feature. The histogram answers "is the model slow?". Your counter answers "what is this feature costing us?" — different questions, different audiences.

```
// Tokens: the meter that bills you
// From the wrapper, at no cost to you:
gen_ai.client.operation.duration // ms
gen_ai.client.token.usage        // by type

// Your own, business-facing counter:
_debriefTokens = meter.CreateCounter<long>(
    "mission.debrief.tokens",
    unit: "{token}");

_debriefTokens.Add(input, shipTag,
    new KeyValuePair<string, object?>(
        "gen_ai.token.type", "input"));
```



Metric output: 3 launches for Enterprise, 1 for Voyager.

The Prompt Is the Riskiest Thing You'll Log

Prompt and completion capture is OFF by default. That default is correct — one boolean turns your observability platform into a searchable copy of everything your users typed.

```
// OFF by default – keep it that way
.UseOpenTelemetry(
    sourceName: "MissionControl.Chat",
    configure: o =>
        o.EnableSensitiveData = false);

// Metadata you still keep either way:
activity.SetTag("gen_ai.usage.input_tokens",
    input);
```



What's actually inside a prompt

- Whatever the user typed
- Retrieved documents (your RAG context)
- System prompts and business rules
- Customer records and case history
- Anything upstream put in context



Safer default: capture token counts, latency, model and finish reason — never the content. **Off in production. On in dev and staging. In production only with redaction you actually built.**



DEMO TIME

AI Mission Debrief

Ask a model to narrate the flight → watch gen_ai spans, token metrics & a correlated log join the very same trace.

gen_ai spans

Token metrics

Correlated logs

No API key



NimblePros

Better software development. Delivered.

GenAI Trace & Token Metrics



The trace

mission.debrief on top — which ship, which outcome. chat gpt-4o-mini underneath, carrying provider, model, finish reason and token counts.



The metrics

gen_ai.client.token.usage split into input and output, plus mission.debrief.tokens grouped by ship. Engineering question and business question.



The logs

Token counts, model and ship on the record. The prompt and the completion are deliberately absent.



One Traceld — ties the button click, the model call, the token counts and the log line into a single story.



Cost math: tokens × your rate card. OTel captures tokens, not dollars — pricing is contractual, so you apply it in the dashboard.

Key Takeaways



OpenTelemetry is the industry standard — vendor-neutral and portable. Switching platforms is a config change, not a rewrite.



Aspire ServiceDefaults makes telemetry reusable. Define once, instrument everywhere — spans and counters.



Logs + Traces + Metrics work together. The TraceId is the thread that connects a log line to the full request path across services.



Your AI calls are just another dependency. The `gen_ai` conventions put the model in the same traces, metrics and logs. Same TraceId, no new vendor.



Logging discipline matters. Structured messages with real attributes, levels used deliberately, and alerts on thresholds — not on every event.



Tokens are a metric with a price. Measure them deliberately — and never log a prompt you have not thought hard about.



Baggage propagates context automatically — great for tenant ID, user ID, feature flags. Never PII.



Thank You, Cloud & AI Summit!

Questions & Discussion

Barret Blake · Architect, NimblePros



Resources

- opentelemetry.io/docs/what-is-opentelemetry/
- opentelemetry.io/docs/languages/dotnet/
- opentelemetry.io — GenAI semantic conventions
- aspire.dev/dashboard/
- github.com/barretb/MissionControlDemo



Contact Me

-  linkedin.com/in/barretblake
-  bluesky: @barretblake.dev
-  barretblake.dev
-  youtube.com/@barretcodes
-  blog.nimblepros.com